

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Инспекция кода»

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Основные методы ручного тестирования	5
2.1 Инспекция кода	5
2.2 Сквозной просмотр кода	6
2.3 Проверка за столом	6
2.4 Контрольные списки	7
2.5 Рецензирование	7
3 Технология инспекции кода	8
3.1 Состав группы	8
3.2 Код программы	8
4 Список вопросов и ответов	13
5 Исправленный код	19
Заключение	24
Список использованной литературы	25

Введение

В процессе разработки программного обеспечения, помимо выполнения основной задачи, заключающейся в реализации функциональности, предусмотренной спецификацией, важной целью является обеспечение высокого качества созданного программного решения.

Тестирование программного обеспечения представляет собой процесс проверки соответствия фактического поведения программы ожидаемому, который осуществляется на основе заранее определенного набора тестов. Однако на практике ни один из методов тестирования не способен выявить абсолютно все дефекты в программном продукте. Это обусловлено ограниченностью ресурсов проекта, таких как бюджет, временные рамки и количество персонала, выделенного на выполнение задач, включая тестирование. Тем не менее, грамотно организованный процесс тестирования позволяет обнаружить большинство дефектов, что способствует их своевременному устранению и, как следствие, повышению качества программного обеспечения.

В рамках данной лабораторной работы применяется метод ручного тестирования, который представляет собой проверку программного обеспечения, осуществляемую специалистами без использования специализированных автоматизированных инструментов.

Следует отметить, что ручное тестирование не заменяет автоматизированное тестирование, а используется в сочетании с ним, что позволяет выявлять дефекты программного обеспечения на более ранних этапах разработки.

1 Постановка задачи

Задача: выполнить инспекцию кода, выступая в роли разработчика программного обеспечения

Для достижения данной цели необходимо выполнить следующие шаги:

1. Ознакомиться с основными методами ручного тестирования
2. Провести анализ кода посредством инспекции, изучив его на наличие ошибок и недочётов
3. Внести необходимые исправления в программу, основываясь на рекомендациях, предоставленных специалистом по тестированию

2 Основные методы ручного тестирования

Ручное тестирование программного обеспечения – это процесс проверки кода и его функциональности без использования автоматизированных инструментов. Несмотря на стремительное развитие технологий и появление мощных средств автоматизированного тестирования, ручные методы остаются важной частью процесса обеспечения качества программного обеспечения. Их применение позволяет обнаружить ошибки на ранних этапах разработки, повысить надежность кода и улучшить взаимодействие внутри команды разработчиков.

2.1 Инспекция кода

Инспекция — это формализованный процесс анализа кода группой специалистов. Задача инспекции — обнаружение ошибок, а не их исправление. Основные этапы инспекции:

1. Подготовка. Координатор раздает участникам листинг программы и спецификацию проекта для предварительного изучения.
2. Заседание.
 - Разработчик объясняет логику работы программы, а остальные участники задают вопросы и анализируют код.
 - Используются контрольные списки распространенных ошибок.
3. Итоги. Все найденные ошибки фиксируются, после чего разработчик исправляет их самостоятельно.

Преимущества:

- Выявление ошибок, связанных с проектированием и кодированием.
- Обучение команды через анализ чужих ошибок и стилей программирования.
- Повышение командного духа и качества взаимодействия.

Недостатки:

- Требуется времени и ресурсов.
- Меньшая эффективность при поиске высокоуровневых ошибок (например, на этапе анализа требований).

2.2 Сквозной просмотр кода

Сквозной просмотр — это менее формализованный процесс, чем инспекция. Группа участников (обычно 3-5 человек) «проигрывает» логику программы, используя тестовые данные. Основные этапы:

1. Подготовка. Участники получают материалы для изучения. Тестировщик готовит набор тестовых данных.
2. Проведение.
 - Тестировщик вводит тестовые данные, а участники вручную отслеживают изменения состояния программы.
 - Задаются вопросы программисту, выявляются ошибки в логике и проектировании.
3. Итоги. Ошибки фиксируются, а разработчик вносит исправления.

Преимущества:

- Участники лучше понимают логику программы.
- Выявляются не только ошибки, но и слабые места программы.

Недостатки:

- Ограниченная скорость проверки из-за ручного анализа.
- Требуется участие нескольких человек, что увеличивает затраты.

2.3 Проверка за столом

Этот метод представляет собой индивидуальный анализ кода программистом. Разработчик самостоятельно вычитывает код, проверяет его с помощью контрольного списка или прогоняет тестовые данные вручную.

Преимущества:

- Простота и доступность.
- Не требует организации группы.

Недостатки:

- Низкая эффективность из-за субъективности и ограниченности одного человека.
- Нарушение принципа тестирования, согласно которому программист не должен проверять собственный код.

2.4 Контрольные списки

Контрольные списки — это инструмент, облегчающий выявление ошибок. Они включают типовые ошибки, которые часто встречаются в коде. Например:

- Обращения к неинициализированным переменным.
- Выход индексов за пределы массива.
- Неправильное использование арифметики целых чисел.
- Ошибки в логике циклов или ветвлений.
- Несоответствие типов данных при передаче параметров между модулями.

Использование таких списков позволяет систематизировать процесс анализа и повысить его эффективность.

2.5 Рецензирование

Рецензирование — это процесс оценки программного кода другими программистами. Цель — получение обратной связи о стиле программирования, удобстве сопровождения и качестве кода. Участники оценивают программы по заранее подготовленным критериям и предоставляют свои замечания.

Преимущества:

- Объективная оценка кода.
- Повышение квалификации программистов через анализ чужих решений.

Недостатки:

- Не направлено на выявление ошибок.
- Требуется участия значительного числа людей для обеспечения анонимности.

3 Технология инспекции кода

3.1 Состав группы

Заседание происходило в следующем составе:

- специалист по тестированию Яшнова Дарья;
- программист Михайлова Алёна
- секретарь Михайлова Алёна

3.2 Код программы

Название: построение двумерного клеточного автомата с окрестностью фон Неймана

Дано:

- *width* - ширина поля
- *height* - высота поля
- *choice* - режим заполнения начального состояния
- *iterations* - число итераций

Требуется: моделирование эволюции клеточного автомата по заданному количеству итераций по заданной функции, где состояние каждой клетки зависит от её соседей (слева, сверху, справа, снизу) и текущего значения.

Ограничения: все параметры - целые числа:

- $width \in [1, 80]$
- $height \in [1, 80]$
- $choice \in (m, r)$
- $iterations \in [1, \infty)$

Спецификация:

Входные данные	Выходные данные	Реакция программы
width = 5, height = 5, choice = r, iterations = 6	Заполненный массив, выведенный на экран с помощью нулей и единиц.	Случайным образом за- полняется поле заданного размера. По заданной функции в программе строится дальнейшее поведение клеточного ав- томата исходя из соседних элементов для каждого элемента поля. На каждом шаге t состояние каждой клетки обновляется со- гласно правилам перехода в зависимости от заданной функции.
width = 5, height = 5, choice = m, iterations = 6	Заполненный массив, выведенный на экран с помощью нулей и единиц.	Пользователем заполняет- ся поле заданного разме- ра. По заданной функции в программе строится даль- нейшее поведение клеточ- ного автомата исходя из со- седних элементов для каж- дого элемента поля. На каждом шаге t состояние каждой клетки обновляет- ся согласно правилам пере- хода в зависимости от за- данной функции.
width = 85	Сообщение на экране: «Ошибка: ширина должна быть целым числом от 1 до 80.»	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение.
width = 3, height = 85	Сообщение на экране: «Ошибка: высота должна быть целым числом от 1 до 80.»	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение.

Исходный код представлен в Листинге 3.2.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <random>
5  #include <iomanip>
6  #include <algorithm>
7
8  using namespace std;
9
10 const string RULE_STRING = "00001001000101100001110100000000";
11

```

```

12 int getNextState(int s0, int s1, int s2, int s3, int s4) {
13     int index = (s0 << 4) | (s1 << 3) | (s2 << 2) | (s3 << 1) |
14         s4;
15     return RULE_STRING[index] - '0';
16 }
17
18 class CellularAutomaton {
19     private:
20     int width, height;
21     vector<vector<int>> currentGrid;
22     vector<vector<int>> nextGrid;
23
24     int toroidal(int x, int size) {
25         return ((x % size) + size) % size;
26     }
27
28     int getNeighbor(int row, int col, int dr, int dc) {
29         int nr = toroidal(row + dr, height);
30         int nc = toroidal(col + dc, width);
31         return currentGrid[nr][nc];
32     }
33
34     public:
35     CellularAutomaton(int w, int h) : width(w), height(h) {
36         currentGrid.resize(height, vector<int>(width, 0));
37         nextGrid.resize(height, vector<int>(width, 0));
38     }
39
40     void setManualInitialState() {
41         cout << "Введите состояние каждой клетки в строке через про
42             бел (0 или 1):\n";
43         for (int i = 0; i < height; ++i) {
44             cout << "Строка " << i + 1 << ": ";
45             for (int j = 0; j < width; ++j) {
46                 cin >> currentGrid[i][j];
47                 if (currentGrid[i][j] != 0 && currentGrid[i][j] != 1) {
48                     cout << "Ошибка: введите только 0 или 1.\n";
49                     j--;
50                 }
51             }
52         }
53     }
54
55     void setRandomInitialState() {
56         random_device rd;
57         mt19937 gen(rd());
58         uniform_int_distribution<> dis(0, 1);
59
60         for (int i = 0; i < height; ++i) {
61             for (int j = 0; j < width; ++j) {
62                 currentGrid[i][j] = dis(gen);
63             }
64         }
65     }
66 }

```

```

62     }
63 }
64
65 void update() {
66     for (int i = 0; i < height; ++i) {
67         for (int j = 0; j < width; ++j) {
68             int s0 = currentGrid[i][j];
69             int s1 = getNeighbor(i, j, -1, 0); // вверх
70             int s2 = getNeighbor(i, j, 0, 1);  // право
71             int s3 = getNeighbor(i, j, 1, 0);  // низ
72             int s4 = getNeighbor(i, j, 0, -1); // лево
73
74             nextGrid[i][j] = getNextState(s0, s1, s2, s3, s4);
75         }
76     }
77     currentGrid = nextGrid;
78 }
79
80 void print() const {
81     for (int i = 0; i < height; ++i) {
82         for (int j = 0; j < width; ++j) {
83             cout << currentGrid[i][j] << " ";
84         }
85         cout << endl;
86     }
87     cout << "-----" << endl;
88 }
89
90 void run(int iterations) {
91     cout << "\nНачальное состояние:\n";
92     print();
93
94     for (int iter = 1; iter <= iterations; ++iter) {
95         cout << "Итерация " << iter << ":\n";
96         update();
97         print();
98     }
99 }
100 };
101
102 int main() {
103     int width, height, iterations;
104     char choice;
105     do {
106         cout << "Введите ширину поля (от 1 до 80): ";
107         cin >> width;
108         if (cin.fail() || width < 1 || width > 80) {
109             cout << "Ошибка: ширина должна быть целым числом от 1 до
110                     80.\n";
111             cin.clear();
112             cin.ignore(10000, '\n');
113             width = -1;

```

```

113     }
114 } while (width < 1 || width > 80);
115 do {
116     cout << "Введите высоту поля (от 1 до 80): ";
117     cin >> height;
118     if (cin.fail() || height < 1 || height > 80) {
119         cout << "Ошибка: высота должна быть целым числом от 1 до
120             80.\n";
121         cin.clear();
122         cin.ignore(10000, '\n');
123         height = -1;
124     }
125 } while (height < 1 || height > 80);
126 CellularAutomaton ca(width, height);
127 do {
128     cout << "Выберите начальные условия:\n";
129     cout << "m - вручную\n";
130     cout << "r - случайно\n";
131     cin >> choice;
132
133     if (cin.fail()) {
134         cin.clear();
135         cin.ignore(10000, '\n');
136         choice = '\0';
137     }
138 } while (choice != 'm' && choice != 'M' && choice != 'r' &&
139     choice != 'R');
140 if (choice == 'm' || choice == 'M') {
141     ca.setManualInitialState();
142 } else {
143     ca.setRandomInitialState();
144 }
145 do {
146     cout << "Введите количество итераций (от 1 и больше): ";
147     cin >> iterations;
148     if (cin.fail() || iterations < 1) {
149         cout << "Ошибка: количество итераций должно быть целым чи
150             слом >= 1.\n";
151         cin.clear();
152         cin.ignore(10000, '\n');
153         iterations = -1;
154     }
155 } while (iterations < 1);
156 ca.run(iterations);
157 return 0;
158 }

```

4 Список вопросов и ответов

1. **Используются ли переменные с неинициализированными значениями?**

Ответ: Нет. Все переменные инициализируются либо при объявлении, либо через валидированный ввод пользователя.

2. **Выходят ли индексы за границы массива?**

Ответ: Нет. Функция `toroidal` обеспечивает корректную обработку границ с тороидальной топологией: `((x % size) + size) % size`.

3. **Используются ли нецелочисленные индексы?**

Ответ: Нет. Все индексы имеют тип `int`.

4. **Вычисляются ли адреса битовых строк? Передаются ли битовые строки в качестве аргументов?**

Ответ: Нет. Строка `RULE_STRING` используется только для доступа по индексу с последующим преобразованием символа в число.

5. **Участвуют ли в вычислениях неарифметические переменные?**

Ответ: Нет. Все вычисления выполняются над целочисленными значениями.

6. **Выполняются ли вычисления с использованием данных разного типа?**

Ответ: Нет. Преобразование `RULE_STRING[index] - '0'` является преднамеренным и корректным.

7. **Выполняются ли вычисления с переменными разной длины?**

Ответ: Нет. Все переменные имеют стандартную длину типа `int`.

8. **Присваиваются ли переменным значения типа данных большего размера?**

Ответ: Нет. Неявных расширений типов, способных вызвать ошибки, не происходит.

9. **Возможны ли переполнение или потеря точности в процессе промежуточных вычислений?**

Ответ: Нет. Максимальное значение индекса в `getNextState` равно 31, что безопасно для типа `int`.

10. **Возможно ли деление на нуль?**

Ответ: Нет. Операции деления в коде отсутствуют.

11. **Критичны ли погрешности, возникающие из-за использования двоичных представлений чисел при вычислениях?**

Ответ: Нет. Вычисления производятся только с целыми числами.

12. Корректно ли используются атрибуты памяти, адресуемой с помощью ссылок и указателей?

Ответ: Да. Явные указатели не используются; память управляется контейнерами `vector`.

13. Согласуются ли описания структуры данных в разных процедурах?

Ответ: Да. `currentGrid` и `nextGrid` последовательно объявлены как `vector<vector<int>`.

14. Не выходят ли значения переменных за логически обоснованные пределы?

Ответ: Нет. Входные данные валидируются: ширина и высота ограничены диапазоном $[1, 80]$, количество итераций ≥ 1 .

15. Понятны ли приоритеты операций?

Ответ: Да. Приоритеты битовых сдвигов и логических операций явно заданы скобками.

16. Существуют ли ошибки завышения или занижения значения индекса на единицу при индексации массивов?

Ответ: Нет. Функция `toroidal` корректно обрабатывает граничные случаи.

17. Соблюдены ли правила наследования объектов?

Ответ: Наследование в коде отсутствует.

18. Все ли переменные объявлены?

Ответ: Да. Все переменные объявлены в соответствующих областях видимости.

19. Понятны ли последствия использования атрибутов по умолчанию?

Ответ: Атрибуты по умолчанию в коде не используются.

20. Есть ли попытки сравнения несопоставимых переменных?

Ответ: Нет. Все сравнения выполняются между совместимыми типами.

21. Есть ли попытки сравнения переменных разного типа?

Ответ: Нет. Все сравнения типизированы корректно.

22. Правильно ли инициализированы массивы и строки?

Ответ: Да. Векторы инициализируются в конструкторе, `RULE_STRING` является константной строкой.

23. Корректно ли используются операторы сравнения?

Ответ: Да.

24. **Правильно ли определены размеры, типы и классы памяти?**
Ответ: Да.
25. **Корректно ли используются булевы выражения?**
Ответ: Да.
26. **Согласуется ли инициализация с классом памяти?**
Ответ: Да.
27. **Корректно ли используются результаты сравнения в булевых выражениях?**
Ответ: Да.
28. **Имеются ли переменные с похожими именами?**
Ответ: Нет. Имена переменных различимы; `s0–s4` намеренно схожи для обозначения соседних клеток.
29. **Корректно ли сравниваются дробные величины?**
Ответ: В коде отсутствуют дробные величины.
30. **Понятны ли приоритеты операций?**
Ответ: Да.
31. **Понятен ли способ разбора булевых выражений компилятором?**
Ответ: Да.
32. **Может ли значение индекса в переключателе превысить число переходов?**
Ответ: Переключатели (`switch`) в коде отсутствуют.
33. **Правильно ли заданы атрибуты файлов?**
Ответ: Код не работает с файлами.
34. **Будет ли завершен каждый цикл?**
Ответ: Да. Все циклы имеют конечные условия; циклы ввода повторяются до получения валидных данных.
35. **Будет ли завершена программа?**
Ответ: Да. Функция `main` возвращает управление после выполнения.
36. **Есть ли цикл, который может не выполниться из-за входных условий?**
Ответ: Нет. Валидация входных данных гарантирует корректные начальные условия для циклов.
37. **Корректны ли инструкции `OPEN`?**
Ответ: Инструкции `OPEN` отсутствуют.

38. **Согласуются ли спецификации формата с инструкциями ввода-вывода?**
Ответ: Да. Используются стандартные потоки `cin/cout`.
39. **Соответствует ли размер буфера размеру записи?**
Ответ: Код не применяет явные буферы.
40. **Корректны ли возможные погружения в цикл?**
Ответ: Да. Вложенные циклы структурированы корректно.
41. **Имеются ли ошибки диапазона, приводящие к отклонению количества итераций от нужного значения?**
Ответ: Нет. Границы циклов заданы корректно.
42. **Открываются ли файлы перед обращением к ним?**
Ответ: Файлы в коде не используются.
43. **Закрываются ли файлы после обращения к ним?**
Ответ: Файлы в коде не используются.
44. **Корректно ли обрабатываются признаки конца файла?**
Ответ: Файлы в коде не используются.
45. **Обрабатываются ли ошибки ввода-вывода?**
Ответ: Да. Ошибки ввода обрабатываются через проверку `cin.fail()` с последующим `cin.clear()` и `cin.ignore()`.
46. **Присутствуют ли в выходной информации орфографические или грамматические ошибки?**
Ответ: Нет.
47. **Существуют ли точки ветвления, в которых не учтены все возможные условия?**
Ответ: Нет. Все ветвления покрывают возможные значения ('m', 'M', 'r', 'R').
48. **Совпадает ли число формальных параметров с числом аргументов?**
Ответ: Да.
49. **Совпадает ли тип возвращаемого значения с типом, объявленным в сигнатуре функций?**
Ответ: Да. Все функции возвращают значения, соответствующие их объявлению.
50. **Совпадают ли атрибуты параметров и аргументов?**
Ответ: Да.

51. **Совпадают ли единицы измерения параметров и аргументов?**
Ответ: Да (единицы измерения не применяются).
52. **Совпадает ли число аргументов, передаваемых вызываемым модулям, с числом ожидаемых параметров?**
Ответ: Да.
53. **Совпадают ли атрибуты аргументов, передаваемых вызываемым модулям, с атрибутами ожидаемых параметров?**
Ответ: Да.
54. **Совпадают ли единицы измерения аргументов, передаваемых вызываемым модулям, с единицами измерения ожидаемых параметров?**
Ответ: Код не использует физические единицы измерения.
55. **Правильно ли заданы количество, атрибуты и порядок следования аргументов при вызове встроенных функций?**
Ответ: Да. Встроенные функции STL используются корректно.
56. **Имеются ли ссылки на параметры, не связанные с текущей точкой входа?**
Ответ: Нет.
57. **Делаются ли в подпрограмме попытки изменить входные аргументы?**
Ответ: Нет. Параметры передаются по значению или константной ссылке.
58. **Согласуются ли определения глобальных переменных в разных модулях?**
Ответ: Модули отсутствуют; глобальная константа `RULE_STRING` используется корректно.
59. **Передаются ли константы в качестве аргументов?**
Ответ: Да, но это не вызывает проблем.
60. **Есть ли в таблице перекрестных ссылок переменные, на которые нет ссылок?**
Ответ: Нет. Все объявленные переменные используются.
61. **Совпадает ли список атрибутов с тем, который следовало ожидать?**
Ответ: Да.
62. **Выдаются ли какие-нибудь предупреждения или информационные сообщения при компиляции?**
Ответ: Нет.

63. Осуществляется ли проверка корректности входных данных?

Ответ: Да. Входные данные (ширина, высота, количество итераций, выбор режима) валидируются с обработкой ошибок ввода.

5 Исправленный код

По результатам заседания в код были внесены следующие изменения:

- Генератор случайных чисел вынесен в приватные члены класса для предотвращения повторной инициализации при каждом вызове `setRandomInitialState`.
- Добавлена проверка длины строки `RULE_STRING` в конструкторе для предотвращения выхода за границы при доступе по индексу.
- Улучшена обработка ввода в методе `setManualInitialState`: добавлена проверка `cin.fail()` для корректной обработки нечислового ввода.

```
#include <iostream>
#include <vector>
#include <string>
#include <random>
#include <algorithm>

using namespace std;

const string RULE_STRING = "00001001000101100001110100000000";

int getNextState(int s0, int s1, int s2, int s3, int s4) {
    int index = (s0 << 4) | (s1 << 3) | (s2 << 2) | (s3 << 1) | s4;
    // Проверка границ для безопасности
    if (index < 0 || index >= static_cast<int>(RULE_STRING.length())) {
        return 0;
    }
    return static_cast<int>(RULE_STRING[index] - '0');
}

class CellularAutomaton {
private:
    int width, height;
    vector<vector<int>> currentGrid;
    vector<vector<int>> nextGrid;
    // Генератор случайных чисел как член класса
    mt19937 randomGenerator;

    int toroidal(int x, int size) const {
        return ((x % size) + size) % size;
    }
}
```

```

int getNeighbor(int row, int col, int dr, int dc) const {
int nr = toroidal(row + dr, height);
int nc = toroidal(col + dc, width);
return currentGrid[nr][nc];
}

public:
CellularAutomaton(int w, int h) : width(w), height(h),
randomGenerator(random_device{}()) {
// Проверка корректности правила
if (RULE_STRING.length() < 32) {
cerr << "Ошибка: строка правила должна содержать минимум 32 символа.\n";
}
currentGrid.resize(height, vector<int>(width, 0));
nextGrid.resize(height, vector<int>(width, 0));
}

void setManualInitialState() {
cout << "Введите состояние каждой клетки в строке через пробел (0 или 1):\n";
for (int i = 0; i < height; ++i) {
cout << "Строка " << i + 1 << ": ";
for (int j = 0; j < width; ++j) {
while (!(cin >> currentGrid[i][j]) ||
(currentGrid[i][j] != 0 && currentGrid[i][j] != 1)) {
cout << "Ошибка: введите только 0 или 1.\n";
cin.clear();
cin.ignore(10000, '\n');
}
}
}
}

void setRandomInitialState() {
uniform_int_distribution<> dis(0, 1);
for (int i = 0; i < height; ++i) {
for (int j = 0; j < width; ++j) {
currentGrid[i][j] = dis(randomGenerator);
}
}
}

void update() {
for (int i = 0; i < height; ++i) {

```

```

for (int j = 0; j < width; ++j) {
    int s0 = currentGrid[i][j];
    int s1 = getNeighbor(i, j, -1, 0); // верх
    int s2 = getNeighbor(i, j, 0, 1);  // право
    int s3 = getNeighbor(i, j, 1, 0);  // низ
    int s4 = getNeighbor(i, j, 0, -1); // лево

    nextGrid[i][j] = getNextState(s0, s1, s2, s3, s4);
}
}
currentGrid = nextGrid;
}

void print() const {
    for (int i = 0; i < height; ++i) {
        for (int j = 0; j < width; ++j) {
            cout << currentGrid[i][j] << " ";
        }
        cout << endl;
    }
    cout << "-----" << endl;
}

void run(int iterations) {
    cout << "\nНачальное состояние:\n";
    print();

    for (int iter = 1; iter <= iterations; ++iter) {
        cout << "Итерация " << iter << ":\n";
        update();
        print();
    }
}

int main() {
    int width, height, iterations;
    char choice;

    do {
        cout << "Введите ширину поля (от 1 до 80): ";
        cin >> width;
        if (cin.fail() || width < 1 || width > 80) {

```

```

cout << "Ошибка: ширина должна быть целым числом от 1 до 80.\n";
cin.clear();
cin.ignore(10000, '\n');
width = -1;
}
} while (width < 1 || width > 80);

```

```

do {
cout << "Введите высоту поля (от 1 до 80): ";
cin >> height;
if (cin.fail() || height < 1 || height > 80) {
cout << "Ошибка: высота должна быть целым числом от 1 до 80.\n";
cin.clear();
cin.ignore(10000, '\n');
height = -1;
}
} while (height < 1 || height > 80);

```

```

CellularAutomaton ca(width, height);

```

```

do {
cout << "Выберите начальные условия:\n";
cout << "m - вручную\n";
cout << "r - случайно\n";
cin >> choice;

```

```

if (cin.fail()) {
cin.clear();
cin.ignore(10000, '\n');
choice = '\0';
}
} while (choice != 'm' && choice != 'M' && choice != 'r' && choice != 'R')

```

```

if (choice == 'm' || choice == 'M') {
ca.setManualInitialState();
} else {
ca.setRandomInitialState();
}

```

```

do {
cout << "Введите количество итераций (от 1 и больше): ";
cin >> iterations;
if (cin.fail() || iterations < 1) {

```

```
cout << "Ошибка: количество итераций должно быть целым числом >= 1.\n";
cin.clear();
cin.ignore(10000, '\n');
iterations = -1;
}
} while (iterations < 1);

ca.run(iterations);
return 0;
}
```

Заключение

В ходе выполнения данной лабораторной работы был изучен метод ручного тестирования – инспекция кода. В проведении инспекционного собрания участвовали: программист, секретарь и специалист по тестированию. В результате инспекции был составлен протокол заседания. Затем мной был проведен анализ составленного протокола, внесены необходимые изменения в код программы.

По итогу выполненной работы, я сделала вывод, что методы ручного тестирования, в частности инспекция кода являются эффективными: мне удалось улучшить читаемость кода и добавить поясняющие комментарии для лучшего понимания кода.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ. – Санкт-Петербург: Диалектика, 2012. – С. 272.